

Assignment 01

Talal jamil(22L-6684)

September 21, 2025

1. Batch Normalization

Concept: Batch Normalization

Explanation: Batch Normalization adjusts layer inputs so they have a mean close to 0 and variance close to 1 within a batch. After normalization, a scaling (γ) and shift (β) are applied. This speeds up training, reduces sensitivity to initialization, and helps prevent gradient issues.

Function:

$$\hat{x} = \frac{x - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}, \quad y = \gamma\hat{x} + \beta$$

Example: If batch mean $\mu_B = 100$ and variance $\sigma_B^2 = 16$, for input $x = 108$:

$$\hat{x} = \frac{108 - 100}{\sqrt{16}} = 2, \quad y = 0.5(2) + 1 = 2$$

2. Vanishing Gradient Problem

Concept: Vanishing Gradient Problem

Explanation: In deep networks with sigmoid/tanh, gradients shrink as they propagate backward. The earlier layers stop receiving meaningful updates, slowing or halting learning.

Function: For sigmoid,

$$\sigma'(x) = \sigma(x)(1 - \sigma(x)) \leq 0.25$$

Example: In a 20-layer network, if each layer passes gradient 0.1:

$$(0.1)^{20} = 1 \times 10^{-20} \quad (\approx 0, \text{no learning})$$

3. Exploding Gradient Problem

Concept: Exploding Gradient Problem

Explanation: When gradients grow too large, weights change drastically, making training unstable. Common in deep or recurrent networks without gradient control.

Function: If gradient ≈ 4 per layer across 12 layers:

$$4^{12} = 16,777,216 \quad (\text{very large})$$

Example: In an RNN across 50 steps, if each step multiplies gradient by 1.5:

$$1.5^{50} \quad (\text{extremely large, causing divergence})$$

4. Leaky ReLU

Concept: Leaky ReLU

Explanation: Leaky ReLU fixes the dead-neuron issue of ReLU by letting negative values pass through with a small slope. This ensures gradients never completely vanish for negative inputs.

Function:

$$f(x) = \begin{cases} x & x > 0 \\ \alpha x & x \leq 0 \end{cases}$$

Example: With $\alpha = 0.05$:

$$f(4) = 4, \quad f(-6) = -0.3$$

5. Dying ReLU Problem

Concept: Dying ReLU Problem

Explanation: A neuron “dies” when its inputs are always negative, outputting 0 constantly. With gradient 0, weights never update, and the neuron becomes inactive.

Function:

$$f(x) = \begin{cases} x & x > 0 \\ 0 & x \leq 0 \end{cases}$$

Example: If inputs are $[-2, -4, -7]$, ReLU outputs $[0, 0, 0]$. The neuron never activates again.

6. Exponential Linear Unit (ELU)

Concept: Exponential Linear Unit (ELU)

Explanation: ELU gives linear outputs for positives and smooth exponential outputs for negatives. It avoids dead neurons and keeps the mean activations closer to zero, improving learning speed.

Function:

$$f(x) = \begin{cases} x & x > 0 \\ \alpha(e^x - 1) & x \leq 0 \end{cases}$$

Example: If $\alpha = 1$:

$$f(2) = 2, \quad f(-1) = e^{-1} - 1 \approx -0.632$$

7. Dropout Regularization

Concept: Dropout Regularization

Explanation: Dropout prevents overfitting by randomly removing some neurons during training. The model can't rely on any single path and learns more generalized features. At testing, all neurons are active but scaled.

Function:

$$y = \begin{cases} 0 & \text{with probability } p \\ \frac{x}{1-p} & \text{with probability } (1-p) \end{cases}$$

Example: With dropout rate $p = 0.3$, hidden activations $[0.6, 0.8, 0.4]$ could become $[0.6, 0, 0.4]$ in one run, and $[0, 0.8, 0]$ in another.

8. 1x1 Convolution (Pointwise Convolution)

Concept: 1x1 Convolution

Explanation: A 1x1 filter slides across all channels of a pixel, combining them into a new set of channels. It changes feature depth but not height/width, useful for dimensionality reduction and channel mixing.

Function:

$$y = \sum_{c=1}^C w_c x_c + b$$

Example: Input channels $[4, 6, 8]$ with weights $[0.1, 0.2, 0.3]$:

$$y = (0.1)(4) + (0.2)(6) + (0.3)(8) = 0.4 + 1.2 + 2.4 = 4$$

9. Residual Connection and Identity Mapping

Concept: Residual Connection and Identity Mapping

Explanation: Instead of learning the full mapping $H(x)$, residual learning makes the network learn only the difference $F(x) = H(x) - x$. A skip connection adds x back:

$$y = F(x) + x$$

Example: If $F(x) = x + 1$ and input $x = 4$:

$$y = (4 + 1) + 4 = 9$$

10. Depthwise Separable Convolution

Concept: Depthwise Separable Convolution

Explanation: It splits a convolution into two steps: (1) Depthwise convolution $\rightarrow$ one filter per channel, (2) Pointwise convolution $\rightarrow 1 \times 1$ filters to combine channel results. This drastically reduces computation while keeping accuracy high.

Function: Standard convolution:

$$D_K^2 \cdot M \cdot N \cdot D_F^2$$

Depthwise separable convolution:

$$D_K^2 \cdot M \cdot D_F^2 + M \cdot N \cdot D_F^2$$

Example: For input $28 \times 28 \times 3$ with 32 filters of size 3×3 :

Standard conv $\approx 652k$ operations

Depthwise separable conv $\approx 92k$ operations